

Homework 4: Pipelining Design

Yousef Awad
EEL 4783 - HDL in Digital Systems

April 4, 2026

1)

Translate the following C code into a Verilog code without pipelining.

```
1 int x = 0;
2 for (int i = 0; i < 3; i++)
3 {
4     x = 2*x + 1;
5 }
```

Unpipelined Verilog Implementation

```
1 module unpipelined (
2     input clk,
3     input reset,
4     input [7:0] x_in,
5     output reg [7:0] x_out
6 );
7     wire [7:0] x1, x2, x3;
8
9     // x = 2x + 1 is effectively (x << 1) + 1
10    assign x1 = (x_in << 1) + 8'd1;
11    assign x2 = (x1 << 1) + 8'd1;
12    assign x3 = (x2 << 1) + 8'd1;
13
14    always @(posedge clk or posedge reset) begin
15        if (reset) x_out <= 8'd0;
16        else x_out <= x3;
17    end
18 endmodule
```

2) Analysis of Unpipelined Design

Based on the implementation in `unpipelined.sv`, the following performance metrics are identified:

- **Throughput:** 8 bits per clock cycle. Since the module produces one 8-bit result every clock cycle, the throughput is constant at 1 result/cycle.
- **Latency:** 1 clock cycle. The data enters the module and the final result of all three iterations is available at the next rising edge of the clock.
- **Timing (Critical Path Delay):** The critical path consists of three consecutive shift-and-add operations.

$$T_{crit} = 3 \times T_{logic} + T_{co} + T_{su}$$

Where T_{logic} is the delay of a single $(2x + 1)$ operation, T_{co} is the clock-to-output delay of the input register, and T_{su} is the setup time of the output register.

3) Pipelined Verilog Code

```
1 module pipelined (
2     input clk,
3     input reset,
4     input [7:0] x_in,
5     output reg [7:0] x_out
6 );
7 reg [7:0] stage1_reg, stage2_reg;
8
9 always @(posedge clk or posedge reset) begin
10     if (reset) begin
11         stage1_reg <= 8'd0;
12         stage2_reg <= 8'd0;
13         x_out <= 8'd0;
14     end else begin
15         // Stage 1: x1 = 2*x_in + 1
16         stage1_reg <= (x_in << 1) + 8'd1;
17         // Stage 2: x2 = 2*stage1 + 1
18         stage2_reg <= (stage1_reg << 1) + 8'd1;
19         // Stage 3: x3 = 2*stage2 + 1
20         x_out <= (stage2_reg << 1) + 8'd1;
21     end
22 end
23 endmodule
```

4. Analysis of Pipelined Design

Based on the implementation in `pipelined.sv`, the following performance metrics are identified:

- **Throughput:** 8 bits per clock cycle. Once the pipeline is full (after the initial latency), the module produces one 8-bit result every clock cycle.
- **Latency:** 3 clock cycles. Since the design is divided into three stages, it takes three clock cycles for a specific input to propagate to the output.
- **Timing (Critical Path Delay):** The critical path is reduced to a single shift-and-add operation between registers.

$$T_{crit} = 1 \times T_{logic} + T_{co} + T_{su}$$

This allows for a significantly higher maximum clock frequency compared to the unpipelined design, as the path delay is approximately one-third of the unpipelined version.